Introduction to Data Science
R exercises in-class
Week 2

Dr. Gah-Yi Ban
Imperial College Business School

Basic Programming in R

Run the following code chunks to get started.

```
# GDP unit : m (million)
uk_GDP_2018 <- 2203005
uk_GDP_2019 <- 2238348
uk_GDP_2020 <- 1991439
uk_GDP_2021 <- 2142738
uk_GDP_2022 <- 2230625
```

```
uk_GDP_growth_rate_2019 <- (uk_GDP_2019 - uk_GDP_2018)/uk_GDP_2018 * 100

print(uk_GDP_growth_rate_2019)
```

```
## [1] 1.604309
```

```
uk_GDP_growth_rate_2020 <- (uk_GDP_2020 - uk_GDP_2019)/uk_GDP_2019 * 100

print(uk_GDP_growth_rate_2020)
```

```
## [1] -11.03086
```

```
uk_GDP_growth_rate_2021 <- (uk_GDP_2021 - uk_GDP_2020)/uk_GDP_2020 * 100

print(uk_GDP_growth_rate_2021)
```

```
## [1] 7.597471
```

Comparison

It is often necessary to compare values and variables, to determine if they are equal, or if one is greater or less than another. The *comparison operators* for equality are `==` and `!=`

```r
# Equal to
uk_GDP_growth_rate_2020 == uk_GDP_growth_rate_2020
```

```
## [1] TRUE
```

```r
# Not equal to
uk_GDP_growth_rate_2020 != uk_GDP_growth_rate_2020
```

```
## [1] FALSE
```

The operators for ordering (greater/less than) are `> < >= <=`

```r
# Less than
uk_GDP_growth_rate_2020 < uk_GDP_growth_rate_2020
```

```
## [1] FALSE
```

```r
# Less than or equal to
uk_GDP_growth_rate_2020 <= uk_GDP_growth_rate_2020
```

```
## [1] TRUE
```

```r
# Greater than
uk_GDP_growth_rate_2020 > uk_GDP_growth_rate_2020
```

```
## [1] FALSE
```

```r
# Greater than or equal to
uk_GDP_growth_rate_2020 >= uk_GDP_growth_rate_2020
```

```
## [1] TRUE
```

Notice that the output is either `TRUE` or `FALSE`. These are *Boolean* values (also called logical values) - a data type, different to numeric or character.

Vector

A vector holds same type of data in an array. A vector can be created using an in-built function in R called `c()`. Elements must be comma-separated. All elements in a vector can be processed all at once; run and study the following code chunk.

```r
# UK average weekly wage between 2018 and 2022, unit: £
uk_avg_weekly_wage <- c(488,    505,    516,    542,    571)

uk_avg_weekly_wage > 500
```

```
## [1] FALSE  TRUE  TRUE  TRUE  TRUE
```

Previously, we converted dates in character type to Date type directly one by one. But with dates in a numeric type vector, we need to convert them into character type before converting into Date type. The following example demonstrates this idea.

```r
# UK average weekly wage between 2018 and 2022, unit: £
uk_avg_weekly_wage <- c(488,    505,    516,    542,    571)

# UK GDP between 2018 and 2022, unit: m£

uk_GDP <- c(2203005, 2238348, 1991439, 2142738, 2230625)

yrs <- c(20010106, 20020406, NA, 20021203, 20031104, 20050908)

yr <- as.Date(as.character(yrs), format = "%Y%m%d")

print(yr)
```

```
## [1] "2001-01-06" "2002-04-06" NA           "2002-12-03" "2003-11-04"
## [6] "2005-09-08"
```

NA values

We can check if there are NA values using the function `is.na()`. Run and study the following code chunk.

```r
# is.na() is a R built-in function for detecting NAs
is.na(yrs)
```

```
## [1] FALSE FALSE  TRUE FALSE FALSE FALSE
```

```r
# which() is for returning the location of TRUE values
which(is.na(yrs))
```

```
## [1] 3
```

Data frame

A data frame is a type of data that uses rows and columns to store tabular data. An important feature of a data frame is that different columns can store different types of data. For example, the following code chunk generates a data frame. After running the code chunk, a data frame is generated and listed in the Environment window (see top right hand side of your screen). Click on the variable name, and a view window will be opened to show the data.

```r
uk_gdp_weekly_wage <- data.frame("Year"=c(2018:2022),
                                 "weekly_wage"=c(488, 505,    516,    542,    571),
                                 "GDP"=
                                   c(2203005, 2238348 ,1991439,   2142738,    NA),
                                 stringsAsFactors = F)

print(uk_gdp_weekly_wage)
```

```
##   Year weekly_wage     GDP
## 1 2018         488 2203005
## 2 2019         505 2238348
## 3 2020         516 1991439
## 4 2021         542 2142738
## 5 2022         571      NA
```

Indexing

Accessing the *elements* of a vector or data frame is done by *indexing* with `[ ]`. The *index* is a position in a vector or data frame, and in R it always starts at 1.

```
# Extract the first element
uk_avg_weekly_wage[1]
```

```
## [1] 488
```

```
# Extract the first three elements
uk_avg_weekly_wage[1:3]
```

```
## [1] 488 505 516
```

```
# Extract the second and fourth elements
uk_avg_weekly_wage[c(2, 4)]
```

```
## [1] 505 542
```

```
# Extract the last element
uk_avg_weekly_wage[length(uk_avg_weekly_wage)]
```

```
## [1] 571
```

```
# Change the first element
uk_avg_weekly_wage[1] <- 4

# Add a new element with append()
uk_avg_weekly_wage <- append(uk_avg_weekly_wage, 10)

print(uk_avg_weekly_wage)
```

```
## [1]   4 505 516 542 571  10
```

Indices with data frame A data frame can be also accessed by the column names and indices. The following example code shows how to access the 2nd element in the Year column with 2 methods.

```
uk_gdp_weekly_wage$Year[2]
```

```
## [1] 2019
```

```
uk_gdp_weekly_wage[2,1]
```

```
## [1] 2019
```

Exercise 1: Indices *Exercise 1* Extract then print the UK GDP and weekly wage data from the year 2021.

```r
uk_gdp_weekly_wage2021 <- uk_gdp_weekly_wage[4,]
print(uk_gdp_weekly_wage2021)
```

```
##   Year weekly_wage     GDP
## 4 2021         542 2142738
```

Exercise 2 Change the weekly wage in 2021 to 550 and print out the latest result.

```r
uk_gdp_weekly_wage2021[1,2] <- 550
print(uk_gdp_weekly_wage2021)
```

```
##   Year weekly_wage     GDP
## 4 2021         550 2142738
```

Exercise 3 Now correct the weekly wage in 2021 back to 542 and print out the latest result

```r
uk_gdp_weekly_wage2021[1,2] <- 542
print(uk_gdp_weekly_wage2021)
```

```
##   Year weekly_wage     GDP
## 4 2021         542 2142738
```

load data

Before starting loading, copy the data file "uk_gdp_and_weeklyWage.csv" to **data** directory in your R **project directory** (i.e., create a new folder called data in the current directory) and run the folllowing code chunk.

```r
uk_gdp_and_weeklyWage <- read.csv("./data/uk_gdp_and_weeklyWage.csv", stringsAsFactors = F)
```

The variable `uk_gdp_and_weeklyWage` stores the data imported. It is a data frame. The argument `stringsAsFactors=F` is used for stopping read.csv from organizing character type of data in factor type (factors are used to work with categorical variables, variables that have a fixed and known set of possible values). The argument is used to avoid some potential troubles.

To view the imported data, simply click on the variable name from the list in the **top-right** Environment panel... **But**, hold on for a second, before clicking, you need to observe the number of rows of data. For example, `uk_gdp_and_weeklyWage` has *23 obs. of 3 variables*, that means, it has 23 rows and 3 columns. If there are 500,000 or more rows or columns, direct clicking will likely freeze your RStudio. In that case, instead of clicking on the variable, the following line is recommended.

```
View(head(uk_gdp_and_weeklyWage))
```

The line can be run in a code chunk and ideally run in the **Console** window (usually, placed at the bottom left). By default, `head` is a built-in function and it retrieves the first 6 rows of data. If more rows are expected, an argument is required. For example,

```r
View(head(uk_gdp_and_weeklyWage, n = 15))
```

Programming Structure

Conditional Statements if. . . else

- Before applying an operation we might verify if it can be executed.
- We might need to apply different operations depending on some conditions.
- We can use an if-else statement to determine what operations need to be executed if a condition is verified.

For example, check if weekly wage in 2021 was more than £550

```
# check which row contains the data of 2021
row_num <- which(uk_gdp_and_weeklyWage$Year == 2021)
if (uk_gdp_and_weeklyWage$Weekly_pay[row_num] > 550) {
  print("Weekly wage in 2021 was more than £550")
} else {
  print("Weekly wage in 2021 was less than £550")
}
```

```
## [1] "Weekly wage in 2021 was less than £550"
```

Exercise 4 By changing the year number, we can check if the weekly wage in 2018 exceeded £550.

```
# check which row contains the data of 2018
row_num <- which(uk_gdp_and_weeklyWage$Year == 2018)
if (uk_gdp_and_weeklyWage$Weekly_pay[row_num] > 550) {
  print("Weekly wage in 2018 was more than £550")
} else {
  print("Weekly wage in 2018 was less than £550")
}
```

```
## [1] "Weekly wage in 2018 was less than £550"
```

Exercise 5 We don't want to change the relevant numbers every time. In the following exercise, use one variable to improve the above code.

```
year_num <- 2018
row_num <- which(uk_gdp_and_weeklyWage$Year == year_num)
if (uk_gdp_and_weeklyWage$Weekly_pay[row_num] > 550) {
  print(paste("Weekly wage in",  year_num, "was more than £550"))
} else {
  print(paste("Weekly wage in", year_num, "was less than £550"))
}
```

```
## [1] "Weekly wage in 2018 was less than £550"
```

loop

What if we want to repeat the same steps many times?

One common way is to use the **for** loop.

- Use **for** loop to repeat the same operations a specific number of times.
- Use indices together with the loop to control element access.

Run the code below and observe what happens.

```
for (i in 1:nrow(uk_gdp_and_weeklyWage)) {
  # nrow returns the number of rows of the data frame
  # 1:nrow() specifies the number of times i varies
  print(uk_gdp_and_weeklyWage$Year[i])
}
```

```
## [1] 2000
## [1] 2001
## [1] 2002
## [1] 2003
## [1] 2004
## [1] 2005
## [1] 2006
## [1] 2007
## [1] 2008
## [1] 2009
## [1] 2010
## [1] 2011
## [1] 2012
## [1] 2013
## [1] 2014
## [1] 2015
## [1] 2016
## [1] 2017
## [1] 2018
## [1] 2019
## [1] 2020
## [1] 2021
## [1] 2022
```

Now based on Exercise 5, code the following:

Exercise 6 Loop through every year in the dataset and check if the weekly wage exceeded £550.

```
for (i in 1:nrow(uk_gdp_and_weeklyWage)) {
  year_num <- uk_gdp_and_weeklyWage$Year[i]
  row_num <- which(uk_gdp_and_weeklyWage$Year == year_num)
  if (uk_gdp_and_weeklyWage$Weekly_pay[row_num] > 550) {
    print(paste("Weekly wage in",  year_num, "was more than £550"))
  } else {
    print(paste("Weekly wage in", year_num, "was less than £550"))
  }
}
```

```
## [1] "Weekly wage in 2000 was less than £550"
## [1] "Weekly wage in 2001 was less than £550"
## [1] "Weekly wage in 2002 was less than £550"
## [1] "Weekly wage in 2003 was less than £550"
## [1] "Weekly wage in 2004 was less than £550"
## [1] "Weekly wage in 2005 was less than £550"
## [1] "Weekly wage in 2006 was less than £550"
## [1] "Weekly wage in 2007 was less than £550"
## [1] "Weekly wage in 2008 was less than £550"
## [1] "Weekly wage in 2009 was less than £550"
## [1] "Weekly wage in 2010 was less than £550"
## [1] "Weekly wage in 2011 was less than £550"
## [1] "Weekly wage in 2012 was less than £550"
## [1] "Weekly wage in 2013 was less than £550"
## [1] "Weekly wage in 2014 was less than £550"
## [1] "Weekly wage in 2015 was less than £550"
## [1] "Weekly wage in 2016 was less than £550"
## [1] "Weekly wage in 2017 was less than £550"
## [1] "Weekly wage in 2018 was less than £550"
## [1] "Weekly wage in 2019 was less than £550"
## [1] "Weekly wage in 2020 was less than £550"
## [1] "Weekly wage in 2021 was less than £550"
## [1] "Weekly wage in 2022 was more than £550"
```

Pipe operators

The pipe operator allows you to send results of one function to another. The %>% is used as pipe function for carrying out a sequences of actions. A set of *verbs* for common data manipulation challenges.

Let's use `tidyverse` package to support `%>%` operators.

NOTE: First install the tidyverse package by running `install.packages("tidyverse")` in the R Console.

```r
library(tidyverse)
```

Before starting any practice and exercises, load the dataset "uk_gdp_and_weeklyWage.csv".

```r
uk_gdp_and_weeklyWage <- read.csv("./data/uk_gdp_and_weeklyWage.csv",
                                  stringsAsFactors = F)
```

With pipe operators, you can use the one or more of the following verbs:

- `select()`: pick variables based on their names.
- `filter()`: pick cases based on their values.
- `mutate()`: add new variables that are functions of existing variables.
- `arrange()` allows you to sort a data frame by a particular column.
- `summarise()`: reduce multiple values down to a single summary.
- These all combine naturally with `group_by()` which allows you to perform any operation "by group".

Note If you use `group()`, then `ungroup()` is always used after performing all the calculations. If you forget to `ungroup()` data, future data management will likely produce errors. Always `ungroup()` swhen you've finished with your calculations.

select

For example, the following code can select the Year column from the data frame.

```
uk_gdp_and_weeklyWage %>% select(Year) %>% head()
```

```
##   Year
## 1 2000
## 2 2001
## 3 2002
## 4 2003
## 5 2004
## 6 2005
```

Exercise 7

Select the Year and weekly wage columns from "uk_gdp_and_weeklyWage".

```
uk_gdp_and_weeklyWage %>% select(Year, Weekly_pay)
```

```
##    Year Weekly_pay
## 1  2000        296
## 2  2001        311
## 3  2002        323
## 4  2003        334
## 5  2004        346
## 6  2005        360
## 7  2006        374
## 8  2007        390
## 9  2008        405
## 10 2009        412
## 11 2010        420
## 12 2011        427
## 13 2012        434
## 14 2013        438
## 15 2014        443
## 16 2015        453
## 17 2016        464
## 18 2017        473
## 19 2018        488
## 20 2019        505
## 21 2020        516
## 22 2021        542
## 23 2022        571
```

If want to store the pipe operations results for later use, assign the operations to a variable.

For example:

```
year_num <- uk_gdp_and_weeklyWage %>% select(Year)
```

filter

To extract a subset of the dataset, `filter` can be used. For example, if want to select the GDP data between 2015 and 2020. We can do

```
uk_gdp_and_weeklyWage %>% select(Year, GDP_m) %>% filter(Year>=2015, Year<=2020)
```

```
##   Year   GDP_m
## 1 2015 2069595
## 2 2016 2114406
## 3 2017 2166073
## 4 2018 2203005
## 5 2019 2238348
## 6 2020 1991439
```

Recall the exercise selecting the subset from leap years last week, it can be implemented using pipe operations.

Exercise 8

Use select and filter with pipe operators to obtain a subset of weekly wage from leap years.

```
uk_gdp_and_weeklyWage %>% select(Year, Weekly_pay) %>%
  filter(Year %% 4 == 0, Year %% 100 != 0 | Year %% 400 ==0 )
```

```
##   Year Weekly_pay
## 1 2000        296
## 2 2004        346
## 3 2008        405
## 4 2012        434
## 5 2016        464
## 6 2020        516
```

mutate

Recall the practice for calculating GDP growth rate in the previous week(s), if we want to calculate all the yearly GDP growth rates from 2000 to 2022, we need to add a column to the data frame and repeat the equation to obtain GDP growth rate results.

One method is to use `loop` to implement the calculation. Read the following code chunk, run it and observe the results.

```
# add a colummn "GDP_growth_rate" to the data frame by assigning NA values
uk_gdp_and_weeklyWage$GDP_growth_rate <- NA

# use loop to calculate the GDP growth rate
# start from the 2nd row because the equation looks backwards one period.
for (i in 2:nrow(uk_gdp_and_weeklyWage)) {
  uk_gdp_and_weeklyWage$GDP_growth_rate[i] <-
    (uk_gdp_and_weeklyWage$GDP_m[i] - uk_gdp_and_weeklyWage$GDP_m[i-1])/
    uk_gdp_and_weeklyWage$GDP_m[i-1] * 100
}
```

`mutate()` creates new columns that are functions of existing variables. It can also modify (if the name is the same as an existing column) and delete columns (by setting their value to NULL).

With `mutate`, `lag()` and pipe operator(s), the above code can be implemented in the following, much smaller, code chunk.

```
uk_gdp_and_weeklyWage <- uk_gdp_and_weeklyWage %>%
  mutate(GDP_growth_rate = (GDP_m - lag(GDP_m)) / lag(GDP_m) * 100)
```

Exercise 9

Use mutate to add a column "weeklyWageLevel" to check if weekly wage was more than 500 or not.

```
uk_gdp_and_weeklyWage %>%
  mutate(weeklyWageLevel = Weekly_pay>500)
```

```
##    Year Weekly_pay   GDP_m GDP_growth_rate weeklyWageLevel
## 1  2000        296 1627447              NA           FALSE
## 2  2001        311 1662558       2.1574282           FALSE
## 3  2002        323 1691998       1.7707653           FALSE
## 4  2003        334 1744840       3.1230533           FALSE
## 5  2004        346 1785756       2.3449715           FALSE
## 6  2005        360 1833406       2.6683377           FALSE
## 7  2006        374 1873015       2.1604053           FALSE
## 8  2007        390 1921029       2.5634605           FALSE
## 9  2008        405 1918064      -0.1543444           FALSE
## 10 2009        412 1831550      -4.5104856           FALSE
## 11 2010        420 1876058       2.4300729           FALSE
## 12 2011        427 1896087       1.0676109           FALSE
## 13 2012        434 1923551       1.4484567           FALSE
## 14 2013        438 1958557       1.8198634           FALSE
## 15 2014        443 2021225       3.1997026           FALSE
## 16 2015        453 2069595       2.3931032           FALSE
## 17 2016        464 2114406       2.1652062           FALSE
## 18 2017        473 2166073       2.4435704           FALSE
## 19 2018        488 2203005       1.7050210           FALSE
## 20 2019        505 2238348       1.6043087            TRUE
## 21 2020        516 1991439     -11.0308585            TRUE
## 22 2021        542 2142738       7.5974710            TRUE
## 23 2022        571 2230625       4.1016214            TRUE
```

arrange

`arrange()` allows you to sort a data frame by a particular column. Try the following code chunk.

```
uk_gdp_and_weeklyWage %>%
  arrange(GDP_growth_rate)
```

```
##    Year Weekly_pay   GDP_m GDP_growth_rate
## 1  2020        516 1991439     -11.0308585
## 2  2009        412 1831550      -4.5104856
## 3  2008        405 1918064      -0.1543444
## 4  2011        427 1896087       1.0676109
```

```
## 5  2012        434 1923551       1.4484567
## 6  2019        505 2238348       1.6043087
## 7  2018        488 2203005       1.7050210
## 8  2002        323 1691998       1.7707653
## 9  2013        438 1958557       1.8198634
## 10 2001        311 1662558       2.1574282
## 11 2006        374 1873015       2.1604053
## 12 2016        464 2114406       2.1652062
## 13 2004        346 1785756       2.3449715
## 14 2015        453 2069595       2.3931032
## 15 2010        420 1876058       2.4300729
## 16 2017        473 2166073       2.4435704
## 17 2007        390 1921029       2.5634605
## 18 2005        360 1833406       2.6683377
## 19 2003        334 1744840       3.1230533
## 20 2014        443 2021225       3.1997026
## 21 2022        571 2230625       4.1016214
## 22 2021        542 2142738       7.5974710
## 23 2000        296 1627447              NA
```

The default is to sort in an ascending order; to sort by a descending order, you need to specify `arrange(desc())`. Try this for "GDP_growth_rate" in the "uk_gdp_and_weeklyWage" data frame.

Exercise 10

```r
uk_gdp_and_weeklyWage %>%
  arrange(desc(GDP_growth_rate))
```

```
##    Year Weekly_pay   GDP_m GDP_growth_rate
## 1  2021        542 2142738       7.5974710
## 2  2022        571 2230625       4.1016214
## 3  2014        443 2021225       3.1997026
## 4  2003        334 1744840       3.1230533
## 5  2005        360 1833406       2.6683377
## 6  2007        390 1921029       2.5634605
## 7  2017        473 2166073       2.4435704
## 8  2010        420 1876058       2.4300729
## 9  2015        453 2069595       2.3931032
## 10 2004        346 1785756       2.3449715
## 11 2016        464 2114406       2.1652062
## 12 2006        374 1873015       2.1604053
## 13 2001        311 1662558       2.1574282
## 14 2013        438 1958557       1.8198634
## 15 2002        323 1691998       1.7707653
## 16 2018        488 2203005       1.7050210
## 17 2019        505 2238348       1.6043087
## 18 2012        434 1923551       1.4484567
## 19 2011        427 1896087       1.0676109
## 20 2008        405 1918064      -0.1543444
## 21 2009        412 1831550      -4.5104856
## 22 2020        516 1991439     -11.0308585
## 23 2000        296 1627447              NA
```

The End